

Unit III
MID-1(1ST HALF)

Section: II CSD-B

Faculty: S Durgadevi

Syllabus:

Decision Trees:

- **Regression Trees**
- **Stopping criteria and Pruning**
- **Categorical Attributes**
- **ID3 and CART.**

Decision Trees:

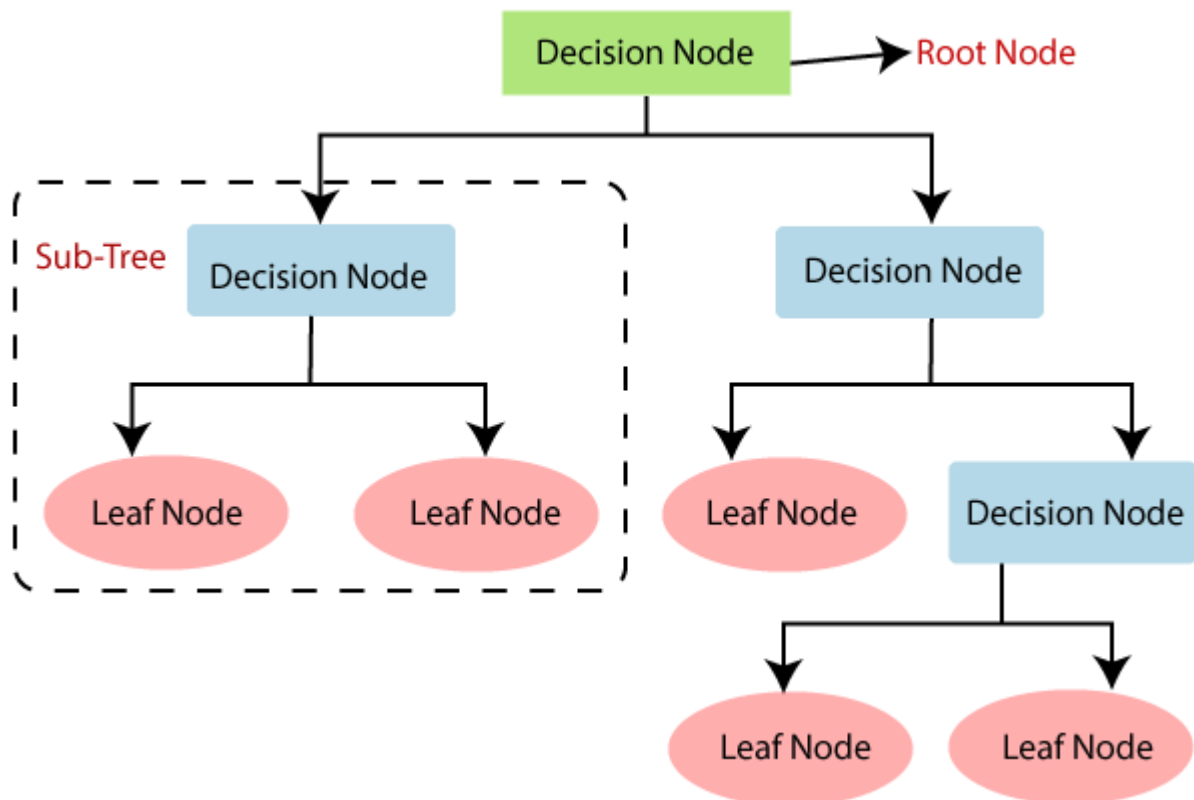
What is a decision tree?

A decision tree is a supervised learning algorithm that is used for classification and regression modeling. Regression is a method used for predictive modeling, so these trees are used to either classify data or predict what will come next.

Decision trees look like flowcharts, starting at the **root node** with a specific question of data, that leads to branches that hold potential answers. The branches then lead to **decision (internal) nodes**, which ask more questions that lead to more outcomes. This goes on until the data reaches what's called a **terminal (or "leaf") node** and ends.

In machine learning, there are four main methods of training algorithms: supervised, unsupervised, reinforcement learning, and semi-supervised learning. A decision tree helps us visualize how a supervised learning algorithm leads to specific outcomes.

- *It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.*
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.
- In order to build a tree, we use the **CART algorithm**, which stands for **Classification and Regression Tree algorithm**.
- A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.
- Below diagram explains the general structure of a decision tree:



Why use Decision Trees?

There are various algorithms in Machine learning, so choosing the best algorithm for the given dataset and problem is the main point to remember while creating a machine learning model. Below are the two reasons for using the Decision tree:

- Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.
- The logic behind the decision tree can be easily understood because it shows a tree-like structure.

Decision Tree Terminologies

- ❑ **Root Node:** Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.
- ❑ **Leaf Node:** Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.
- ❑ **Splitting:** Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.
- ❑ **Branch/Sub Tree:** A tree formed by splitting the tree.

□ **Pruning:** Pruning is the process of removing the unwanted branches from the tree.

□ **Parent/Child node:** The root node of the tree is called the parent node, and other nodes are called the child nodes.

How does the Decision Tree algorithm Work?

In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.

For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm:

- **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.
- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM)**.
- **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
- **Step-4:** Generate the decision tree node, which contains the best attribute.
- **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Attribute Selection Measures

While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes. So, to solve such problems there is a technique which is called as **Attribute selection measure or ASM**. By this measurement, we can easily select the best attribute for the nodes of the tree. There are two popular techniques for ASM, which are:

- **Information Gain**

- **Gini Index**

1. Information Gain:

- Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute.
- It calculates how much information a feature provides us about a class.
- According to the value of information gain, we split the node and build the decision tree.
- A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first. It can be calculated using the below formula:

$$1. \text{ Information Gain} = \text{Entropy}(S) - [(\text{Weighted Avg}) * \text{Entropy}(\text{each feature})]$$

Entropy: Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data. Entropy can be calculated as:

$$\text{Entropy}(s) = -P(\text{yes}) \log_2 P(\text{yes}) - P(\text{no}) \log_2 P(\text{no})$$

Where,

- **S= Total number of samples**
- **P(yes)= probability of yes**
- **P(no)= probability of no**

2. Gini Index:

- Gini index is a measure of impurity or purity used while creating a decision tree in the CART(Classification and Regression Tree) algorithm.
- An attribute with the low Gini index should be preferred as compared to the high Gini index.
- It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits.
- Gini index can be calculated using the below formula:

$$\text{Gini Index} = 1 - \sum_j P_j^2$$

Pruning: Getting an Optimal Decision tree

Pruning is a process of deleting the unnecessary nodes from a tree in order to get the optimal decision tree.

A too-large tree increases the risk of overfitting, and a small tree may not capture all the important features of the dataset. Therefore, a technique that decreases the size of the learning tree without reducing accuracy is known as Pruning. There are mainly two types of tree **pruning** technology used:

- **Cost Complexity Pruning**
- **Reduced Error Pruning.**

Advantages of the Decision Tree

- It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- It can be very useful for solving decision-related problems.
- It helps to think about all the possible outcomes for a problem.
- There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree

- The decision tree contains lots of layers, which makes it complex.
- It may have an overfitting issue, which can be resolved using the **Random Forest algorithm**.
- For more class labels, the computational complexity of the decision tree may increase.

Python Implementation of Decision Tree

Now we will implement the Decision tree using Python. For this, we will use the dataset "**user_data.csv**," which we have used in previous classification models. By using the same dataset, we can compare the Decision tree classifier with other classification models such as [KNN SVM](#), [LogisticRegression](#), etc.

Steps will also remain the same, which are given below:

- **Data Pre-processing step**

- **Fitting a Decision-Tree algorithm to the Training set**
- **Predicting the test result**
- **Test accuracy of the result(Creation of Confusion matrix)**
- **Visualizing the test set result.**

1. Regression Trees:

In machine learning, regression trees are a powerful tool for predicting continuous numerical values. These trees help us model the relationship between a dependent (target) variable and one or more independent (predictor) variables.

What Is Regression Analysis?

Regression analysis is a statistical method that allows us to understand how the value of a dependent variable changes in response to changes in one or more independent variables.

It's commonly used for predicting continuous real-world values such as temperature, age, salary, and prices.

Imagine a marketing company that runs various advertisements each year and wants to predict sales based on their ad spending. Regression analysis helps solve such prediction problems.

Regression Analysis in Machine learning

Regression analysis is a statistical method to model the relationship between a dependent (target) and independent (predictor) variables with one or more independent variables. More specifically, Regression analysis helps us to understand how the value of the dependent variable is changing corresponding to an independent variable when other independent variables are held fixed. It predicts continuous/real values such as **temperature, age, salary, price**, etc.

We can understand the concept of regression analysis using the below example:

Example: Suppose there is a marketing company A, who does various advertisement every year and get sales on that. The below list shows the advertisement made by the company in the last 5 years and the corresponding sales:

Advertisement	Sales
\$90	\$1000
\$120	\$1300
\$150	\$1800
\$100	\$1200
\$130	\$1380
\$200	??

Now, the company wants to do the advertisement of \$200 in the year 2019 **and wants to know the prediction about the sales for this year**. So to solve such type of prediction problems in machine learning, we need regression analysis.

Regression is a supervised learning technique which helps in finding the correlation between variables and enables us to predict the continuous output variable based on the one or more predictor variables. It is mainly used for **prediction, forecasting, time series modeling, and determining the causal-effect relationship between variables**.

"Regression shows a line or curve that passes through all the datapoints on target-predictor graph in such a way that the vertical distance between the datapoints and the regression line is minimum." The distance between datapoints and line tells whether a model has captured a strong relationship or not.

Some examples of regression can be as:

- Prediction of rain using temperature and other factors
- Determining Market trends
- Prediction of road accidents due to rash driving.

Terminologies Related to the Regression Analysis:

- **Dependent Variable:** The main factor in Regression analysis which we want to predict or understand is called the dependent variable. It is also called **target variable**.
- **Independent Variable:** The factors which affect the dependent variables or which are used to predict the values of the dependent variables are called independent variable, also called as a **predictor**.
- **Outliers:** Outlier is an observation which contains either very low value or very high value in comparison to other observed values. An outlier may hamper the result, so it should be avoided.
- **Multicollinearity:** If the independent variables are highly correlated with each other than other variables, then such condition is called Multicollinearity. It should not be present in the dataset, because it creates problem while ranking the most affecting variable.
- **Underfitting and Overfitting:** If our algorithm works well with the training dataset but not well with test dataset, then such problem is called **Overfitting**. And if our algorithm does not perform well even with training dataset, then such problem is called **underfitting**.

Why do we use Regression Analysis?

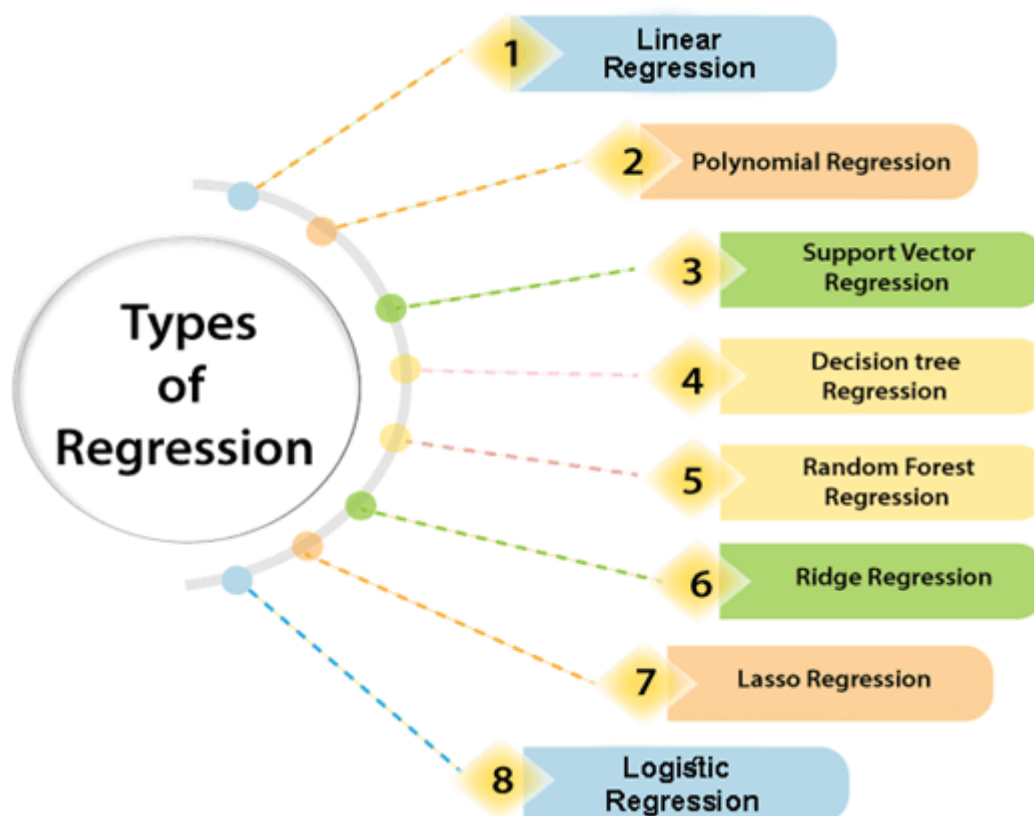
Regression analysis helps in the prediction of a continuous variable. There are various scenarios in the real world where we need some future predictions such as weather condition, sales prediction, marketing trends, etc., for such case we need some technology which can make predictions more accurately. So for such case we need Regression analysis which is a statistical method and used in machine learning and data science. Below are some other reasons for using Regression analysis:

- Regression estimates the relationship between the target and the independent variable.
- It is used to find the trends in data.
- It helps to predict real/continuous values.
- By performing the regression, we can confidently determine the **most important factor, the least important factor, and how each factor is affecting the other factors**.

Types of Regression

There are various types of regressions which are used in data science and machine learning. Each type has its own importance on different scenarios, but at the core, all the regression methods analyze the effect of the independent variable on dependent variables. Here we are discussing some important types of regression which are given below:

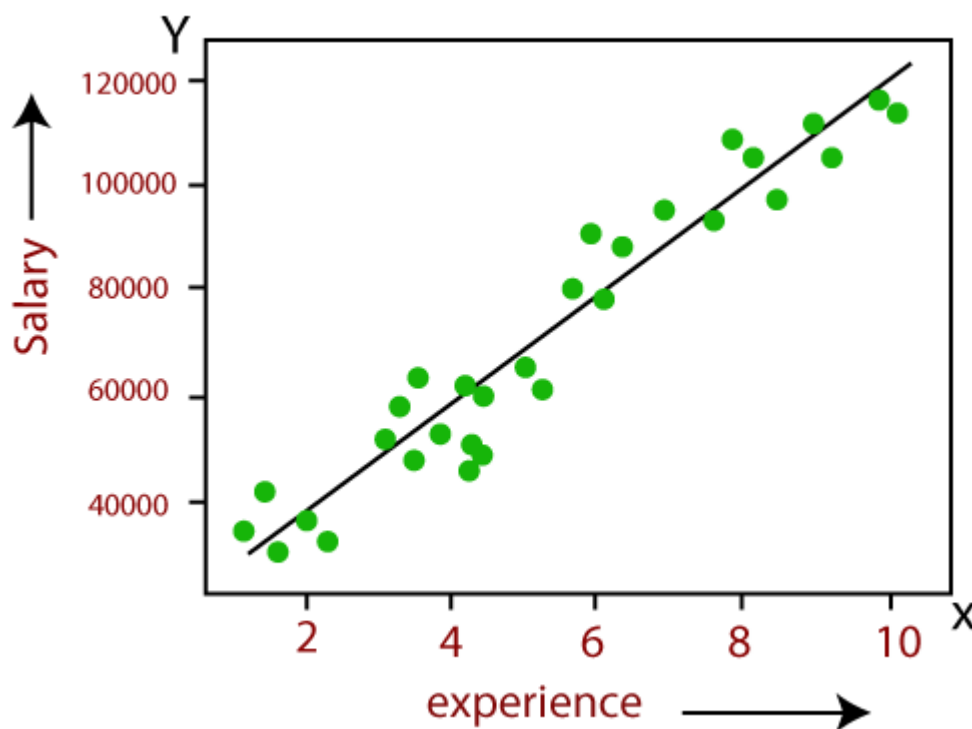
- **Linear Regression**
- **Logistic Regression**
- **Polynomial Regression**
- **Support Vector Regression**
- **Decision Tree Regression**
- **Random Forest Regression**
- **Ridge Regression**
- **Lasso Regression:**



Linear Regression:

- Linear regression is a statistical regression method which is used for predictive analysis.

- It is one of the very simple and easy algorithms which works on regression and shows the relationship between the continuous variables.
- It is used for solving the regression problem in machine learning.
- Linear regression shows the linear relationship between the independent variable (X-axis) and the dependent variable (Y-axis), hence called linear regression.
- If there is only one input variable (x), then such linear regression is called **simple linear regression**. And if there is more than one input variable, then such linear regression is called **multiple linear regression**.
- The relationship between variables in the linear regression model can be explained using the below image. Here we are predicting the salary of an employee on the basis of **the year of experience**.



- Below is the mathematical equation for Linear regression:

1. $Y = aX + b$

Here, Y = dependent variables (target variables),
 X = Independent variables (predictor variables),
 a and b are the linear coefficients

Some popular applications of linear regression are:

- **Analyzing trends and sales estimates**
- **Salary forecasting**
- **Real estate prediction**
- **Arriving at ETAs in traffic.**

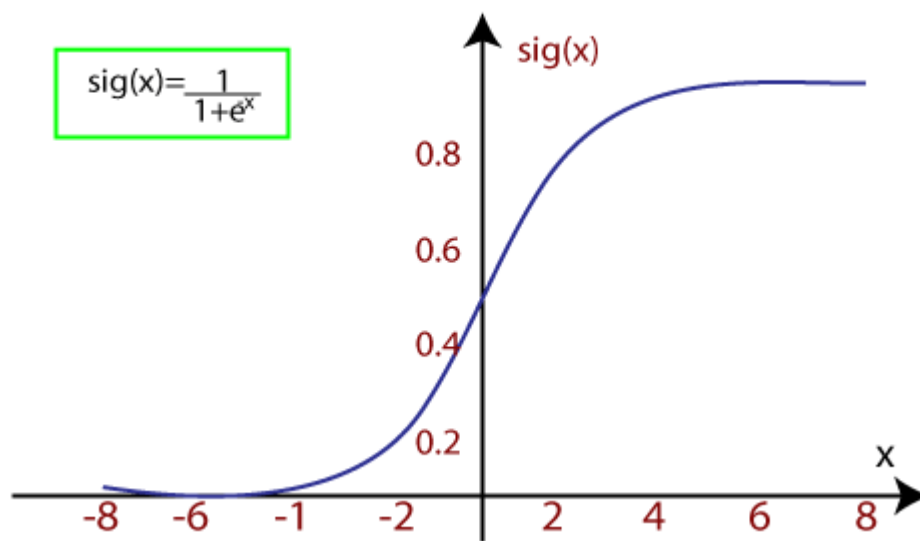
Logistic Regression:

- Logistic regression is another supervised learning algorithm which is used to solve the classification problems. In **classification problems**, we have dependent variables in a binary or discrete format such as 0 or 1.
- Logistic regression algorithm works with the categorical variable such as 0 or 1, Yes or No, True or False, Spam or not spam, etc.
- It is a predictive analysis algorithm which works on the concept of probability.
- Logistic regression is a type of regression, but it is different from the linear regression algorithm in the term how they are used.
- Logistic regression uses **sigmoid function** or logistic function which is a complex cost function. This sigmoid function is used to model the data in logistic regression. The function can be represented as:

$$f(x) = \frac{1}{1 + e^{-x}}$$

- $f(x)$ = Output between the 0 and 1 value.
- x = input to the function
- e = base of natural logarithm.

When we provide the input values (data) to the function, it gives the S-curve as follows:



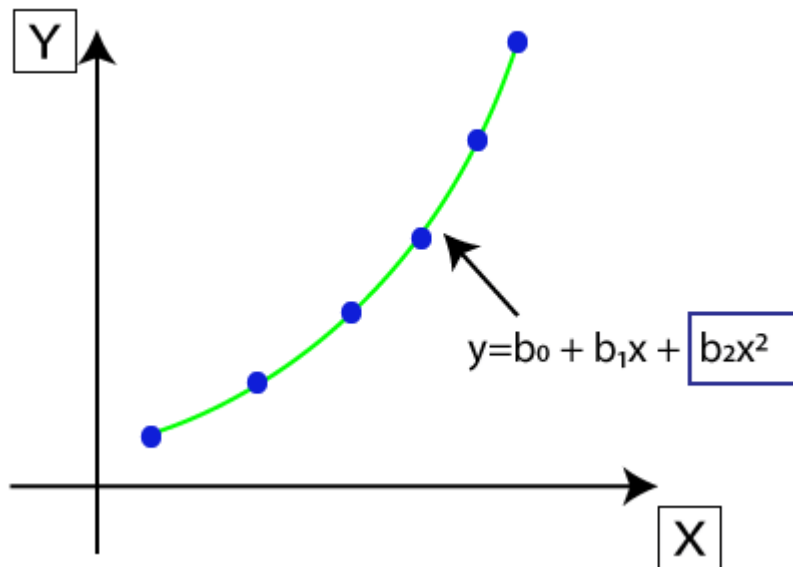
- It uses the concept of threshold levels, values above the threshold level are rounded up to 1, and values below the threshold level are rounded up to 0.

There are three types of logistic regression:

- **Binary(0/1, pass/fail)**
- **Multi(cats, dogs, lions)**
- **Ordinal(low, medium, high)**

Polynomial Regression:

- Polynomial Regression is a type of regression which models the **non-linear dataset** using a linear model.
- It is similar to multiple linear regression, but it fits a non-linear curve between the value of x and corresponding conditional values of y.
- Suppose there is a dataset which consists of datapoints which are present in a non-linear fashion, so for such case, linear regression will not best fit to those datapoints. To cover such datapoints, we need Polynomial regression.
- **In Polynomial regression, the original features are transformed into polynomial features of given degree and then modeled using a linear model.** Which means the datapoints are best fitted using a polynomial line.



- The equation for polynomial regression also derived from linear regression equation that means Linear regression equation $Y = b_0 + b_1x$, is transformed into Polynomial regression equation $Y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots + b_nx^n$.
- Here Y is the **predicted/target output**, $b_0, b_1, \dots, b_n$ are the **regression coefficients**. x is our **independent/input variable**.
- The model is still linear as the coefficients are still linear with quadratic

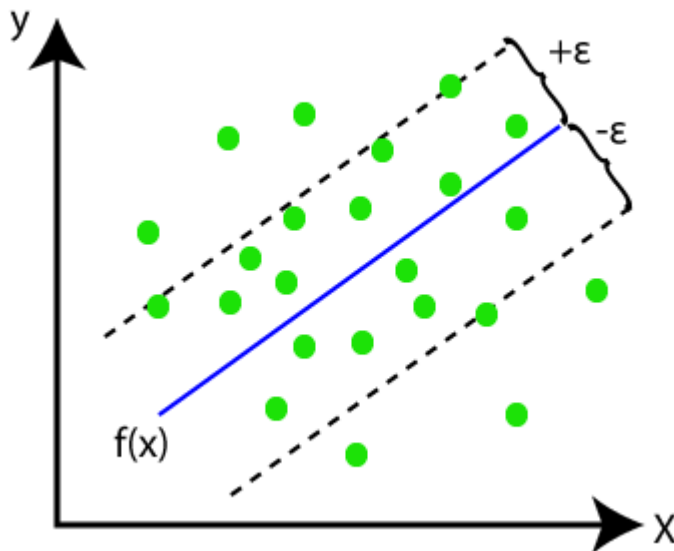
Support Vector Regression:

Support Vector Machine is a supervised learning algorithm which can be used for regression as well as classification problems. So if we use it for regression problems, then it is termed as Support Vector Regression.

Support Vector Regression is a regression algorithm which works for continuous variables. Below are some keywords which are used in **Support Vector Regression**:

- **Kernel:** It is a function used to map a lower-dimensional data into higher dimensional data.
- **Hyperplane:** In general SVM, it is a separation line between two classes, but in SVR, it is a line which helps to predict the continuous variables and cover most of the datapoints.
- **Boundary line:** Boundary lines are the two lines apart from hyperplane, which creates a margin for datapoints.
- **Support vectors:** Support vectors are the datapoints which are nearest to the hyperplane and opposite class.

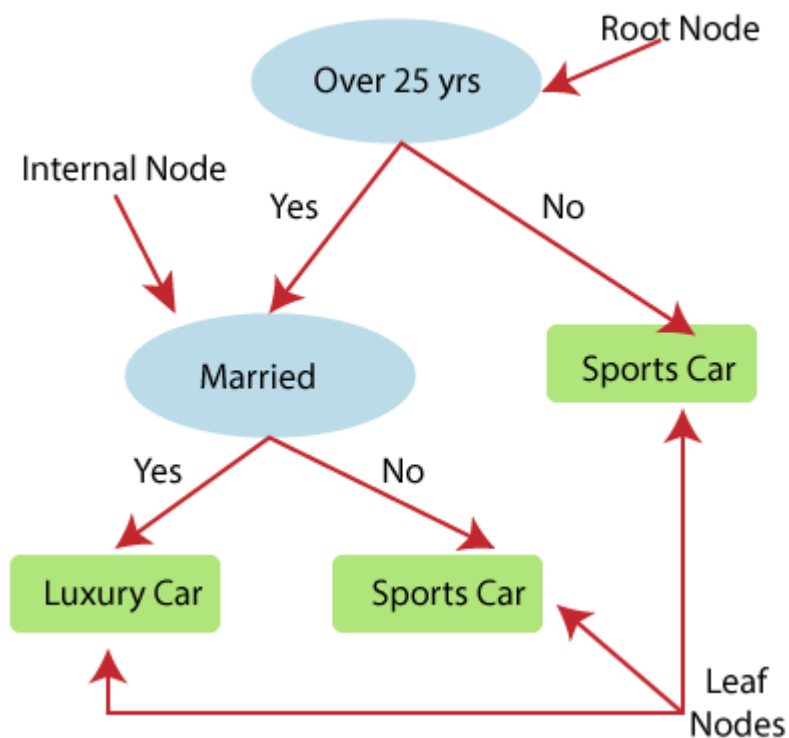
In SVR, we always try to determine a hyperplane with a maximum margin, so that maximum number of datapoints are covered in that margin. ***The main goal of SVR is to consider the maximum datapoints within the boundary lines and the hyperplane (best-fit line) must contain a maximum number of datapoints.*** Consider the below image:



Here, the blue line is called hyperplane, and the other two lines are known as boundary lines.

Decision Tree Regression:

- Decision Tree is a supervised learning algorithm which can be used for solving both classification and regression problems.
- It can solve problems for both categorical and numerical data
- Decision Tree regression builds a tree-like structure in which each internal node represents the "test" for an attribute, each branch represent the result of the test, and each leaf node represents the final decision or result.
- A decision tree is constructed starting from the root node/parent node (dataset), which splits into left and right child nodes (subsets of dataset). These child nodes are further divided into their children node, and themselves become the parent node of those nodes. Consider the below image:

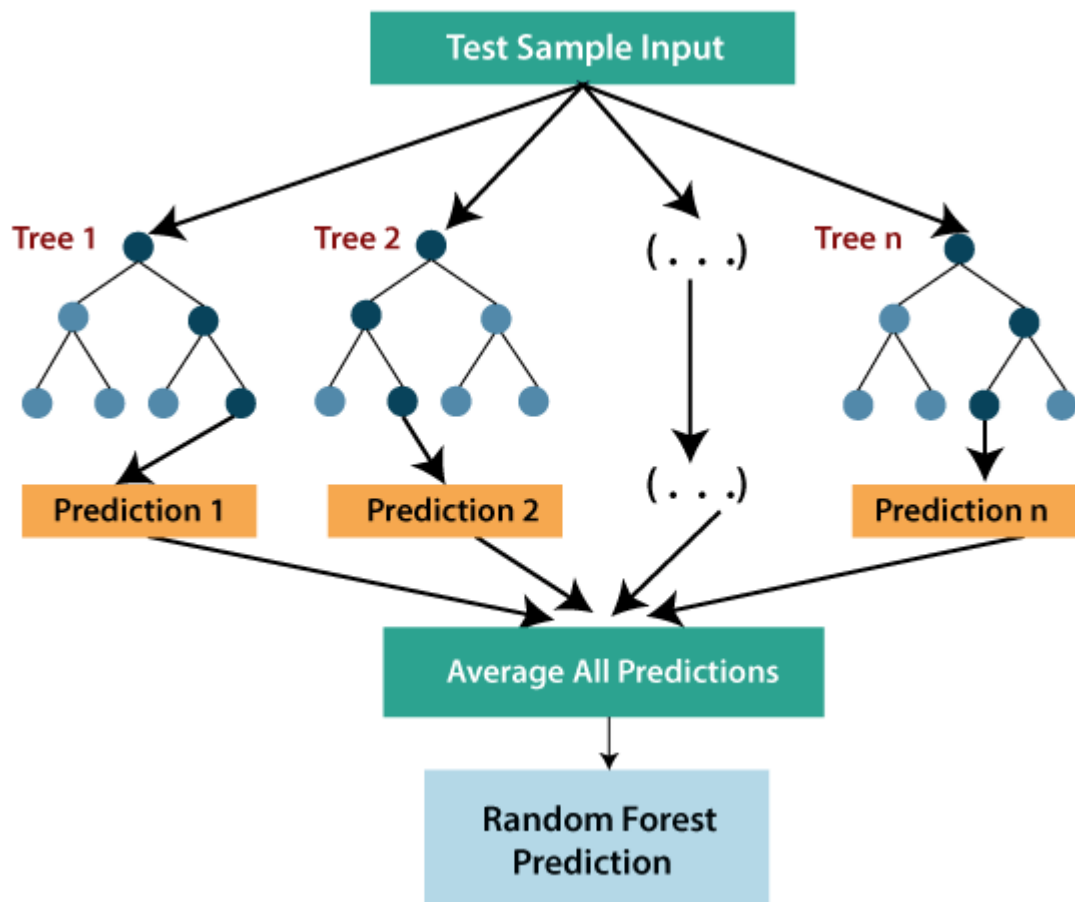


Above image showing the example of Decision Tree regression, here, the model is trying to predict the choice of a person between Sports cars or Luxury car.

- Random forest is one of the most powerful supervised learning algorithms which is capable of performing regression as well as classification tasks.
- The Random Forest regression is an ensemble learning method which combines multiple decision trees and predicts the final output based on the average of each tree output. The combined decision trees are called as base models, and it can be represented more formally as:

$$g(x) = f_0(x) + f_1(x) + f_2(x) + \dots$$

- Random forest uses **Bagging or Bootstrap Aggregation** technique of ensemble learning in which aggregated decision tree runs in parallel and do not interact with each other.
- With the help of Random Forest regression, we can prevent Overfitting in the model by creating random subsets of the dataset.



Ridge Regression:

- Ridge regression is one of the most robust versions of linear regression in which a small amount of bias is introduced so that we can get better long term predictions.
- The amount of bias added to the model is known as **Ridge Regression penalty**. We can compute this penalty term by multiplying with the lambda to the squared weight of each individual features.
- The equation for ridge regression will be:

$$L(x, y) = \text{Min}(\sum_{i=1}^n (y_i - w_i x_i)^2 + \lambda \sum_{i=1}^n (w_i)^2)$$

- A general linear or polynomial regression will fail if there is high collinearity between the independent variables, so to solve such problems, Ridge regression can be used.
- Ridge regression is a regularization technique, which is used to reduce the complexity of the model. It is also called as **L2 regularization**.
- It helps to solve the problems if we have more parameters than samples.

Lasso Regression:

- Lasso regression is another regularization technique to reduce the complexity of the model.
- It is similar to the Ridge Regression except that penalty term contains only the absolute weights instead of a square of weights.
- Since it takes absolute values, hence, it can shrink the slope to 0, whereas Ridge Regression can only shrink it near to 0.
- It is also called as **L1 regularization**. The equation for Lasso regression will be:

$$L(x, y) = \text{Min} \left(\sum_{i=1}^n (y_i - w_i x_i)^2 + \lambda \sum_{i=1}^n |w_i| \right)$$

2. Stopping Criteria and Pruning:

2.1 Stopping Criteria

- Stop growing the tree when further splits give less than some minimal amount of extra information.
- Stop growing the tree when further splits would result in nodes containing less than, say, five percent of the total data.
- Stop growing the tree when the number of cases in the node is less than some pre-specified limit.
- Stop growing the tree when the purity of the node is more than some pre-specified limit.
- Stop growing the tree when the depth of the node is more than some pre-specified limit.
- Stop growing the tree when predictor values for all records are identical, in which no rule could be generated to split them.

Stopping criteria in regression trees are conditions that determine when the tree-building process should stop, thereby preventing overfitting and ensuring that the tree generalizes well to unseen data.

1. Gini Index:

- The Gini index is a widely used measure of impurity or inequality in a group of samples within the context of decision trees.
- It quantifies how effectively a specific attribute divides the data into homogeneous subsets.

- The Gini index ranges from 0 to 1, where 0 represents a pure set (all samples belong to the same class), and 1 indicates maximum impurity (equal distribution of samples among classes).
- When constructing a decision tree, the attribute with the **lowest Gini index** is chosen as the optimal attribute for data splitting.

2. Information Gain:

- Information gain is another criterion for assessing the quality of a split.
- It measures the reduction in entropy (uncertainty) achieved by dividing the data based on a specific attribute.
- Entropy for a set S is calculated as the sum of $(p * \log_2(p))$, where p represents the likelihood of each class in the set.
- The information gain is determined by subtracting the weighted average entropy of child subsets from the entropy of the parent set S following the split.
- The attribute that results in the highest information gain is selected for data splitting.

Here are some common stopping criteria used in regression tree algorithms:

- ❖ **Minimum Node Size:** This criterion involves setting a minimum number of samples required to split a node further. If a node contains fewer samples than this threshold, splitting it will not be allowed. This helps prevent the algorithm from creating very small, specific branches that may overfit the training data.
- ❖ **Maximum Tree Depth:** This criterion limits the maximum depth or levels of the tree. Once the tree reaches this depth, further splitting is not allowed. Limiting the depth helps control the complexity of the tree and prevents it from growing excessively deep, which can lead to overfitting.
- ❖ **Minimum Impurity Decrease:** Impurity measures like Gini impurity or entropy are used to determine the quality of a split. This criterion specifies a minimum improvement in impurity that must be achieved to split a node. If the improvement is below this threshold, the split is not performed, preventing overly specific splits that capture noise in the data.
- ❖ **Maximum Number of Leaf Nodes:** Instead of limiting the depth of the tree, this criterion limits the total number of leaf nodes (terminal nodes) in

the tree. Once this limit is reached, further splitting is stopped, even if the tree is not fully grown. This helps control the complexity of the tree and can be useful when dealing with large datasets.

- ❖ **Variance Threshold:** In regression trees, another criterion could involve setting a minimum reduction in variance that must be achieved to perform a split. This is particularly relevant in regression problems where the goal is to minimize the variance of the target variable within each leaf node.

2.2 Pruning

Pruning is a crucial technique associated with decision trees in machine learning.

1. Decision Trees :

Decision trees are widely used in machine learning. They work by recursively finding the best predictor variable to split the data into subsets, aiming to partition the target outcome more effectively.

However, decision trees are prone to overfitting, where they memorize training data noise along with the signal.

2. Pruning Techniques:

Pruning helps mitigate overfitting by reducing the size of decision trees. It involves removing parts of the tree that do not significantly contribute to classifying instances.

There are two main pruning techniques:

Pre-Pruning (Early Stopping): Stop the tree before it fully classifies the training set. This prevents excessive growth and overfitting.

Post-Pruning: Prune the tree after it has finished building. This involves cutting back the tree to improve its generalization.

3. Pre-pruning

Pre-pruning involves stopping the tree-building process early, before it becomes fully grown. This can be achieved by setting stopping criteria such as:

Maximum tree depth: Limiting the maximum depth of the tree helps prevent overfitting by controlling how many splits are made.

Minimum samples per leaf node: Specifying a minimum number of samples required to be present in each leaf node can prevent the creation of nodes with very few samples, which might lead to overfitting.

Minimum impurity decrease: Nodes are only split if the split results in a decrease in impurity beyond a certain threshold, thus preventing splits that don't contribute much to the model's performance.

4. Post-Pruning:

Post-pruning involves growing the tree to its maximum size and then selectively removing nodes or branches that do not provide significant predictive power. This is usually done using techniques like:

- ❖ **Cost-complexity pruning (also known as weakest link pruning or reduced-error pruning):** This method involves iteratively collapsing nodes with the least influence on reducing the overall error of the tree. It uses a cost-complexity parameter (often denoted as α) to control the trade-off between tree complexity and accuracy. By varying α , a sequence of subtrees is generated, and the optimal subtree is selected based on cross-validation or validation set performance.
- ❖ **Error-based pruning:** Nodes or subtrees are pruned based on the estimated error rate on a separate validation dataset. Subtrees that do not significantly improve predictive accuracy are pruned from the tree.

Key points:

- After constructing the tree (and without early stopping), it may be overfitted.
- The CART algorithm repeatedly partitions data into smaller subsets until the final leaves consist of only a few data points.
- Pruning involves cutting back the tree by removing unnecessary branches (leaves) that don't enhance classification power.
- The goal is to strike a balance between model complexity and accuracy.

5. How Post-Pruning Works:

- **Hold-out test set:** The fastest and simplest method is to evaluate the effect of removing each leaf node using a hold-out test set.
- **Remove nodes:** If removing a leaf node improves the overall cost function on the entire test set, prune it.
- **Stop when no further improvements:** Continue removing nodes until no further improvements can be made¹.

3. Categorical Attributes

categorical attributes are variables that represent distinct categories rather than continuous numerical values. For example, "color" with categories like "red," "green," and "blue" is a categorical attribute.

When dealing with categorical attributes in regression trees, here's how it's typically handled:

1. **Binary Splitting:** This approach involves splitting the dataset based on whether each instance has a particular category or not. For instance, if the attribute is "color" and the categories are "red" and "not red," the dataset would be divided accordingly.
2. **One-Hot Encoding:** This technique converts each category into a binary attribute. For instance, "color" with categories "red," "green," and "blue" would become three binary attributes: "is_red," "is_green," and "is_blue." Each instance then gets a 1 or 0 for each attribute, indicating whether it belongs to that category or not.
3. **Label Encoding:** Categorical attributes can be encoded with numerical labels, but this might not be suitable for regression trees because the order or magnitude of the labels may not reflect the relationship between categories.
4. **Decision Based on Information Gain or Similar Criteria:** The algorithm selects the attribute that maximizes information gain or minimizes variance when deciding where to split the data. This decision is based on the attribute's ability to reduce uncertainty or variance in the target variable.
5. **Handling Missing Values:** Dealing with missing values in categorical attributes is important. It can be treated as a separate category or imputed before building the tree.

Each approach has its pros and cons, and the choice depends on factors like the nature of the data and the specific requirements of the regression task. Experimentation is often needed to determine the most effective method for a particular dataset.

4.ID3 and CART.

4.1 ID3 algorithm

The ID3 (Iterative Dichotomiser 3) algorithm is a popular method for constructing decision trees, a type of supervised learning algorithm used for

classification tasks. Developed by Ross Quinlan in 1986, ID3 is one of the earliest and simplest algorithms for decision tree construction.

ID3 Algorithm

A well-known decision tree approach for machine learning is the Iterative Dichotomiser 3 (ID3) algorithm. By choosing the best characteristic at each node to partition the data depending on information gain, it recursively constructs a tree. The goal is to make the final subsets as homogeneous as possible. By choosing features that offer the greatest reduction in entropy or uncertainty, ID3 iteratively grows the tree. The procedure keeps going until a halting requirement is satisfied, like a minimum subset size or a maximum tree depth.

How ID3 Works

The ID3 algorithm is specifically designed for building [decision trees](#) from a given dataset. Its primary objective is to construct a tree that best explains the relationship between attributes in the data and their corresponding class labels.

1. Selecting the Best Attribute

- ID3 employs the concept of entropy and information gain to determine the attribute that best separates the data. Entropy measures the impurity or randomness in the dataset.
- The algorithm calculates the entropy of each attribute and selects the one that results in the most significant information gain when used for splitting the data.

2. Creating Tree Nodes

- The chosen attribute is used to split the dataset into subsets based on its distinct values.
- For each subset, ID3 recurses to find the next best attribute to further partition the data, forming branches and new nodes accordingly.

3. Stopping Criteria

- The recursion continues until one of the stopping criteria is met, such as when all instances in a branch belong to the same class or when all attributes have been used for splitting.

4. Handling Missing Values

- ID3 can handle missing attribute values by employing various strategies like attribute mean/mode substitution or using majority class values.

5. Tree Pruning

- Pruning is a technique to prevent overfitting. While not directly included in ID3, post-processing techniques or variations like C4.5 incorporate pruning to improve the tree's generalization.

Objectives:

Knowing the basics of the ID3 Algorithm

Loading csv data in python, (using pandas library)

Training and building Decision tree using ID3 algorithm from scratch

Predicting from the tree

Finding out the accuracy

Step 1: Observing The dataset

A	B	C	D	E	F
Outlook	Temperature	Humidity	Wind	Play Tennis	
Sunny	Hot	High	Weak	No	
Sunny	Hot	High	Strong	No	
Overcast	Hot	High	Weak	Yes	
Rain	Mild	High	Weak	Yes	
Rain	Cool	Normal	Weak	Yes	
Rain	Cool	Normal	Strong	No	
Overcast	Cool	Normal	Strong	Yes	
Sunny	Mild	High	Weak	No	
Sunny	Cool	Normal	Weak	Yes	
Rain	Mild	Normal	Weak	Yes	
Sunny	Mild	Normal	Strong	Yes	
Overcast	Mild	High	Strong	Yes	
Overcast	Hot	Normal	Weak	Yes	
Rain	Mild	High	Strong	No	

After observing the dataset we can say that:

Features: Outlook, Temperature, Humidity, Wind

Label: Play Tennis (The output feature that we want to predict)

Class: Yes, No (Unique values of the label)

Step 2: Importing the necessary basic python libraries

use pandas for manipulating the dataset and numpy library for mathematical calculation.

```
1 import pandas as pd #for manipulating the csv data
2 import numpy as np #for mathematical calculation
```

Step 3: Reading the dataset

We are going to read the dataset (csv file) and load it into pandas dataframe.

You can see below, *train_data_m* is our dataframe.

With the *head()* method of the dataframe we can view the first 5 rows.

The output will be:

	Outlook	Temperature	Humidity	Wind	Play Tennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes

Step 4: Calculating the entropy of the whole dataset

In mathematics, we had to calculate the entropy of the whole dataset first like below:

Total row = 14

Row with "Yes" class = 9

Row with "No" class = 5 Complete entropy of dataset is -

$$\begin{aligned} H(S) &= -p(\text{Yes}) * \log_2(p(\text{Yes})) - p(\text{No}) * \log_2(p(\text{No})) \\ &= - (9/14) * \log_2(9/14) - (5/14) * \log_2(5/14) \\ &= - (-0.41) - (-0.53) \\ &= 0.94 \end{aligned}$$

Method description:

Calculates the entropy of the whole dataset.

train_data: a pandas dataframe

label: string, name of the label of the dataframe (=Play Tennis)

class_list: list, unique classes of the label ([Yes, No]).

returns: float, calculated entropy of the whole dataframe (=0.94)

Step 5: Calculating the entropy for the filtered dataset

Using the table of step 1, The mathematics calculation will be like below for the feature **Outlook**:

Categorical values of Outlook - Sunny, Overcast and Rain

Total count of row containing:

Sunny = 5

Sunny & Yes = 2

Sunny & No = 3 >> $H(\text{Outlook}=\text{Sunny}) = -(2/5)*\log(2/5)-(3/5)*\log(3/5) = 0.971$

Total count of row containing:

Rain = 5

Rain & Yes = 3

Rain & No = 2 >> $H(\text{Outlook}=\text{Rain}) = -(3/5)*\log(3/5)-(2/5)*\log(2/5) = 0.971$ Total

count of row containing:

Overcast = 4

Overcast & Yes = 4

Overcast & No = 0 >> $H(\text{Outlook}=\text{Overcast}) = -(4/4)*\log(4/4)-0 = 0$

Method description:

Calculates entropy of a specific feature = value.

feature_value_data: a pandas dataframe, which contains data that has a specific value of a feature (Ex. Only data with Outlook = Sunny)

label: string, name of the label of the dataframe (=Play Tennis)

class_list: list, unique classes of the label ([Yes, No]).

returns: float, calculated entropy of the feature value dataframe (Ex. for Outlook = Sunny, returns 0.971)

Step 6: Calculating information gain for a feature

After calculating entropy, we have to calculate the information gain of that feature. In math, first, we have to calculate the information of that feature like this: (for the feature **Outlook**)

$$I(\text{Outlook}) = p(\text{Sunny}) * H(\text{Outlook}=\text{Sunny}) + p(\text{Rain}) * H(\text{Outlook}=\text{Rain}) + p(\text{Overcast}) * H(\text{Outlook}=\text{Overcast})$$
$$= (5/14)*0.971 + (5/14)*0.971 + (4/14)*0$$
$$= 0.693$$

Then, we have to subtract this from the total entropy of the dataset which is the information gain of the feature.

$$\begin{aligned}\text{Information Gain} &= H(S) - I(\text{Outlook}) \\ &= 0.94 - 0.693 \\ &= 0.247\end{aligned}$$

In python we have done like this:

Method description:

Calculates information gain of a feature.

feature_name: string, the name of the feature that we want to find the information gain (Ex. *Outlook*)

train_data: a pandas dataframe

label: string, name of the label of the dataframe (= *Play Tennis*)

class_list: list, unique classes of the label (= [*Yes*, *No*]).

returns: calculated information gain of the feature (Ex. for *Outlook*, returns 0.247)

Step 7: Finding the most informative feature (feature with highest information gain)

Like **Outlook** feature, We have to calculate information gain for every feature in the dataset. Then we have to select the feature with the highest information gain. After calculating mathematically we will find the values like below:

Information gain:

Outlook = 0.247 (Highest value)

Temperature = 0.0292

Humidity = 0.153

Wind = 0.048

Method description:

Finds the most informative feature from the current dataset.

train_data: a pandas dataframe

label: string, name of the label of the dataframe (=Play Tennis)

class_list: list, unique classes of the label (=[Yes, No]).

returns: string, the feature name

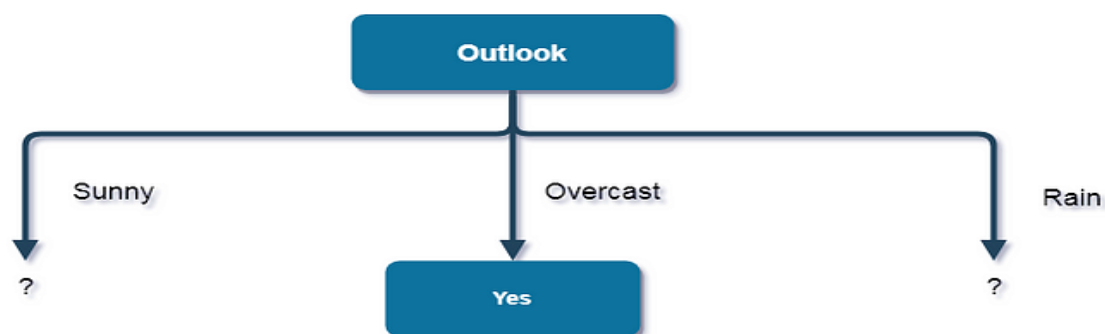
Step 8 : Adding a node to the tree

Outlook is selected as Node.

(Outlook = Sunny): Not pure class, contains both class Yes and No

(Outlook = Overcast): Pure class, contains only one class Yes

(Outlook = Rain): Not pure class, contains both class Yes and No



After selecting a **pure class***, we have to remove the rows from the dataset corresponding to the feature value. Ex: we have to remove rows with **Outlook** = *Overcast*. The resultant dataset will be like below:

A	B	C	D	E	
Outlook	Temperature	Humidity	Wind	Play Tennis	
Sunny	Hot	High	Weak	No	
Sunny	Hot	High	Strong	No	
Rain	Mild	High	Weak	Yes	
Rain	Cool	Normal	Weak	Yes	
Rain	Cool	Normal	Strong	No	
Sunny	Mild	High	Weak	No	
Sunny	Cool	Normal	Weak	Yes	
Rain	Mild	Normal	Weak	Yes	
Sunny	Mild	Normal	Strong	Yes	
Rain	Mild	High	Strong	No	

Step 9: Performing ID3 Algorithm and generating Tree

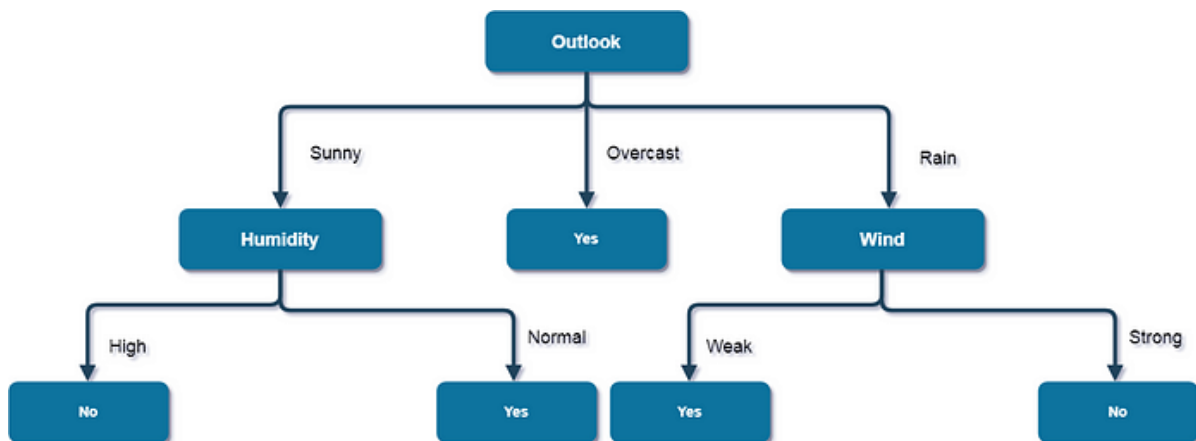
- Finding the most informative feature
- Making a tree node with a feature name and feature values as branches
 - If pure class, adding leaf node (= Class) to the tree node
 - If impure class, adding an expandable node (= '?') to the tree node
- Shrinking/Updating the dataset according to the pure class
- Adding the node with branches into a tree
- Expand the branch of the next impure class (= '?') with an updated dataset

The recursion endpoint:

- The dataset becomes empty after updating
- There is no expandable branch (= all pure class)

Step 10: Finding unique classes of the label and Starting the algorithm

First, we need to see what should be the expected tree:



Method description:

Generates id3 tree.

train_data_m: a pandas dataframe

label: string, name of the label of the dataframe (=Play Tennis)

returns: (nested) dictionary, the decision tree

Step 11: Predicting from the tree

Method description:

Predicts from the generated tree using the feature set/instance

tree: dictionary (of dictionaries), a decision tree

instance: a row or snapshot or set of the features of dataset. The row may not contain label

returns: Any datatype (Int or Float or String etc.) depending on the datatype of the class, the predicted class

Step 12: Evaluating test dataset

To evaluate the model i.e. the tree we need a labeled test dataset. Then after predicting we can calculate the difference of actual and predicted value in percentage.

Method description:

Evaluates the accuracy of a id3 tree by testing against the expected result

tree: dictionary (of dictionaries), a decision tree

test_data_m: a pandas dataframe/test dataset

returns: float, the accuracy of the tree

Step 13: Checking test dataset and Evaluating it

The test dataset looks like below:

A	B	C	D	E	F
Outlook	Temperatu	Humidity	Wind	Play Tennis	
Overcast	Mild	Normal	Weak	Yes	
Sunny	Mild	Normal	Weak	Yes	
Rain	Mild	Normal	Weak	Yes	
Rain	Cool	High	Strong	No	

Link of the test dataset: <https://www.kaggle.com/tareqjoy/testplaytennis>

Here, the Play Tennis label acts as expected output which will be used to find the difference from the predicted output.

4.2 CART ALGORITHM

CART(Classification And Regression Trees) is a variation of the decision tree algorithm. It can handle both [classification and regression](#) tasks. [Scikit-Learn](#) uses the Classification And Regression Tree (CART) algorithm to train [Decision Trees](#) (also called “growing” trees). CART was first produced by Leo Breiman, Jerome Friedman, Richard Olshen, and Charles Stone in 1984.

CART is a predictive algorithm used in [Machine learning](#) and it explains how the target variable's values can be predicted based on other matters. It is a decision tree where each fork is split into a predictor variable and each node has a prediction for the target variable at the end.

The term CART serves as a generic term for the following categories of decision trees:

- **Classification Trees:** The tree is used to determine which “class” the target variable is most likely to fall into when it is continuous.
- **Regression trees:** These are used to predict a continuous variable's value.

CART Algorithm

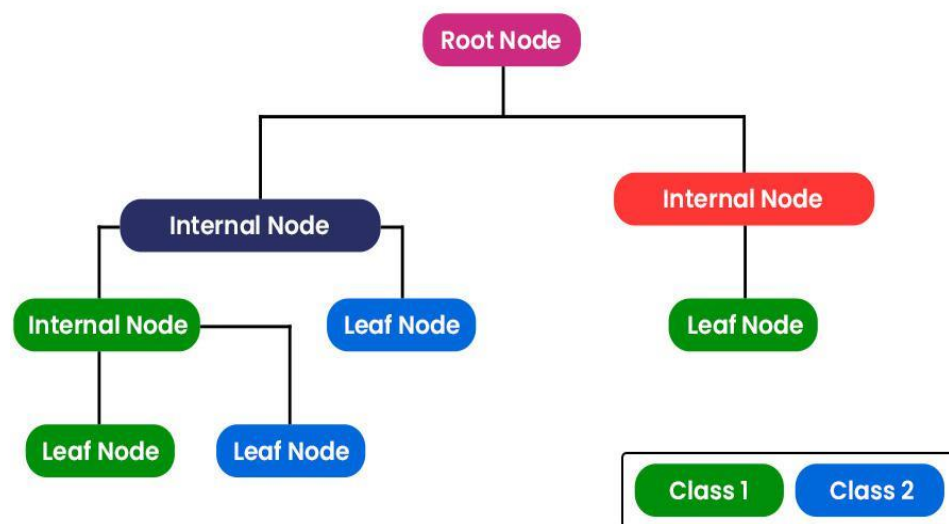
Classification and Regression Trees (CART) is a decision tree algorithm that is used for both classification and regression tasks. It is a supervised learning algorithm that learns from labelled data to predict unseen data.

- **Tree structure:** CART builds a tree-like structure consisting of nodes and branches. The nodes represent different decision points, and the branches represent the possible outcomes of those decisions. The leaf nodes in the tree contain a predicted class label or value for the target variable.
- **Splitting criteria:** CART uses a greedy approach to split the data at each node. It evaluates all possible splits and selects the one that best reduces the impurity of the resulting subsets. For classification tasks, CART uses Gini impurity as the splitting criterion. The lower the Gini impurity, the more pure the subset is. For regression tasks, CART uses residual reduction as the splitting criterion. The lower the residual reduction, the better the fit of the model to the data.
- **Pruning:** To prevent overfitting of the data, pruning is a technique used to remove the nodes that contribute little to the model accuracy. Cost complexity pruning and information gain pruning are two popular pruning techniques. Cost complexity pruning involves calculating the cost of each node and removing nodes that have a negative cost. Information gain pruning involves calculating the information gain of each node and removing nodes that have a low information gain.

CART algorithm works?

The CART algorithm works via the following process:

- The best-split point of each input is obtained.
- Based on the best-split points of each input in Step 1, the new “best” split point is identified.
- Split the chosen input according to the “best” split point.
- Continue splitting until a stopping rule is satisfied or no further desirable splitting is available.



CART algorithm uses Gini Impurity to split the dataset into a decision tree. It does that by searching for the best homogeneity for the sub nodes, with the help of the Gini index criterion.

CART for Classification

A classification tree is an algorithm where the target variable is categorical. The algorithm is then used to identify the “Class” within which the target variable is most likely to fall. Classification trees are used when the dataset needs to be split into classes that belong to the response variable (like yes or no).

For [classification](#) in decision tree learning algorithm that creates a tree-like structure to predict class labels. The tree consists of nodes, which represent

different decision points, and branches, which represent the possible result of those decisions. Predicted class labels are present at each leaf node of the tree.

How Does CART for Classification Work?

CART for classification works by recursively splitting the training data into smaller and smaller subsets based on certain criteria. The goal is to split the data in a way that minimizes the impurity within each subset. Impurity is a measure of how mixed up the data is in a particular subset. For classification tasks, CART uses Gini impurity

- **Gini Impurity-** Gini impurity measures the probability of misclassifying a random instance from a subset labeled according to the majority class. Lower Gini impurity means more purity of the subset.
- **Splitting Criteria-** The CART algorithm evaluates all potential splits at every node and chooses the one that best decreases the Gini impurity of the resultant subsets. This process continues until a stopping criterion is reached, like a maximum tree depth or a minimum number of instances in a leaf node.

CART for Regression

A Regression tree is an algorithm where the target variable is continuous and the tree is used to predict its value. Regression trees are used when the response variable is continuous. For example, if the response variable is the temperature of the day.

CART for [regression](#) is a decision tree learning method that creates a tree-like structure to predict continuous target variables. The tree consists of nodes that represent different decision points and branches that represent the possible outcomes of those decisions. Predicted values for the target variable are stored in each leaf node of the tree.

How Does CART works for Regression?

Regression CART works by splitting the training data recursively into smaller subsets based on specific criteria. The objective is to split the data in a way that minimizes the residual reduction in each subset.

- **Residual Reduction-** Residual reduction is a measure of how much the average squared difference between the predicted values and the actual values for the target variable is reduced by splitting the subset. The lower the residual reduction, the better the model fits the data.
- **Splitting Criteria-** CART evaluates every possible split at each node and selects the one that results in the greatest reduction of residual error in the resulting subsets. This process is repeated until a stopping criterion is met, such as reaching the maximum tree depth or having too few instances in a leaf node.

CART model representation

CART models are formed by picking input variables and evaluating split points on those variables until an appropriate tree is produced.

Steps to create a Decision Tree using the CART algorithm:

- **Greedy algorithm:** In this The input space is divided using the Greedy method which is known as a recursive binary splitting. This is a numerical method within which all of the values are aligned and several other split points are tried and assessed using a cost function.
- **Stopping Criterion:** As it works its way down the tree with the training data, the recursive binary splitting method described above must know when to stop splitting.
- **Tree pruning:** Decision tree's complexity is defined as the number of splits in the tree. Trees with fewer branches are recommended as they are simple to grasp and less prone to cluster the data
- **Data preparation for the CART:** No special data preparation is required for the CART algorithm.

Advantages of CART

- Results are simplistic.
- Classification and regression trees are Nonparametric and Nonlinear.
- Classification and regression trees implicitly perform feature selection.
- Outliers have no meaningful effect on CART.

- It requires minimal supervision and produces easy-to-understand models.

Limitations of CART

- Overfitting.
- High Variance.
- low bias.
- the tree structure may be unstable.

Applications of the CART algorithm

- For quick Data insights.
- In Blood Donors Classification.
- For environmental and ecological data.
- In the financial sectors.